

Part A — Theory

1. Introduction to Regression

What is regression in Machine Learning?

Regression is a supervised learning technique used to predict continuous numerical values by learning relationships between input variables (features) and an output variable (target). It helps understand how changes in one or more factors influence a measurable outcome and is widely used in forecasting, risk analysis, decision-making and trend estimation.

Regression analysis is a statistical tool for the investigation of relationships between variables. Usually, the investigator seeks to ascertain the causal effect of one variable upon another—the effect of a price increase upon demand.

Regression is a statistical technique used to study the relationship between independent and dependent variables. In machine learning, regression analysis is a fundamental concept that consists of a set of machine learning methods that predict a continuous outcome variable (y) based on the value of one or multiple predictor variables (x).

As a result, it helps establish a relationship between the variables by estimating how one variable affects the other.

How is it different from classification?

To understand how machine learning models make predictions, it's important to know the difference between Classification and Regression. Both are supervised learning techniques, but they solve different types of problems depending on the nature of the target variable.

- Classification predicts categories or labels like spam/not spam, disease/no disease, etc.
- Regression predicts continuous values like price, temperature, sales, etc.

Classification uses a decision boundary to separate data into classes, while regression fits a line through continuous data points to predict numerical values

Example of Linear Regression in Real Life

You might have read lot of tutorials on Linear Regression and already have the assumption - Linear Regression is not easy to Understand. We will make Linear Regression very easy for you. Let's boil down each concept and learn with help of Examples. If you have no idea what Linear regression is, this tutorial will be help you understand the basics.

Linear regression might sound like a complex term, but it's actually a very simple concept. Linear Regression is all about finding patterns in data. When two things are connected, (like - **hours of study and test scores**, OR **temperature and ice cream sales**) linear regression helps us understand and predict how one affects the other.

Basically, Linear Regression is asking if *Thing-1 will change, how Thing-2 will respond?* Answer of this question is often found by drawing a straight line through data points on a graph.

Understand Linear Regression with Help of Example

The more time you spend in studying, the better your test scores. Linear regression helps us find the relationship between these two things and use that relationship to make predictions.

Think of it like this:

- You collect some data: how many hours you studied and the scores you got on tests.
- You plot this data on a graph.
- Then, you draw a straight line through the points in such a way that it's as close as possible to all the points. This line shows the trend.

Once you have this line, you can use it to make predictions. For example, if you studied 5 hours for a test, the line can help you estimate what score you're likely to get.

2. Types of Regression

Regression Analysis is a fundamental concept in machine learning used to model relationships between dependent and independent variables. Various regression techniques are tailored to different data structures and objectives. Below is an exploration of key regression techniques, their significance, and practical examples.

Types of Regression Techniques

1. [Linear Regression](#)
2. [Polynomial Regression](#)
3. [Stepwise Regression](#)
4. [Decision Tree Regression](#)
5. [Random Forest Regression](#)
6. [Support Vector Regression](#)
7. [Ridge Regression](#)
8. [Lasso Regression](#)
9. [ElasticNet Regression](#)
10. [Bayesian Linear Regression](#)

1. Linear Regression

Linear regression is used for predictive analysis. [Linear regression](#) is a linear approach for modeling the relationship between the criterion or the scalar response and the multiple predictors or explanatory variables. Linear regression focuses on the conditional probability distribution of the response given the values of the predictors. For linear regression, there is a danger of [overfitting](#). The formula for linear regression is:

Syntax:

$$y = \theta x + b$$

where,

- θ - It is the model weights or parameters
- b - It is known as the bias.

This is the most basic form of regression analysis and is used to model a linear relationship between a single dependent variable and one or more independent variables.

Here, a linear regression model is instantiated to fit a linear relationship between input features (X) and target values (y).

2. Polynomial Regression

This is an extension of linear regression and is used to model a non-linear relationship between the dependent variable and independent variables. Here as well syntax remains the same but now in the input variables we include some polynomial or higher degree terms of some already existing features as well. Linear regression was only able to fit a linear model to the data at hand but with [polynomial features](#), we can easily fit some non-linear relationship between the target as well as input features

3. Stepwise Regression

[Stepwise regression](#) is used for fitting regression models with predictive models. It is carried out automatically. With each step, the variable is added or subtracted from the set of explanatory variables. The approaches for stepwise regression are forward selection, backward elimination, and bidirectional elimination.

4. Decision Tree Regression

A Decision Tree is the most powerful and popular tool for classification and prediction. A [Decision tree](#) is a flowchart-like tree structure, where each internal node denotes a test on an attribute, each branch represents an outcome of the test, and each leaf node (terminal node) holds a class label. There is a non-parametric method used to model a decision tree to predict a continuous outcome.

5. Random Forest Regression

Random Forest is an [ensemble](#) technique capable of performing both regression and classification tasks with the use of multiple decision trees and a technique called Bootstrap and Aggregation, commonly known as [bagging](#). The basic idea behind this is to combine multiple decision trees in determining the final output rather than relying on individual decision trees.

[Random Forest](#) has multiple decision trees as base learning models. We randomly perform row sampling and feature sampling from the dataset forming sample datasets for every model. This part is called Bootstrap.

. Support Vector Regression (SVR)

[Support vector regression \(SVR\)](#) is a type of [support vector machine \(SVM\)](#) that is used for regression tasks. It tries to find a function that best predicts the continuous output value for a given input value.

SVR can use both linear and non-linear kernels. A linear kernel is a simple dot product between two input vectors, while a non-linear kernel is a more complex function that can capture more intricate patterns in the data. The choice of kernel depends on the data's characteristics and the task's complexity.

7. Ridge Regression

[Ridge regression](#) is a technique for analyzing multiple regression data. When multicollinearity occurs, least squares estimates are unbiased. This is a regularized linear regression model, it tries to reduce the model complexity by adding a penalty term to the cost function. A degree of bias is added to the regression estimates, and as a result, ridge regression reduces the standard errors.

8. Lasso Regression

[Lasso regression](#) is a regression analysis method that performs both variable selection and [regularization](#). Lasso regression uses soft thresholding. Lasso regression selects only a subset of the provided covariates for use in the final model.

This is another regularized linear regression model, it works by adding a penalty term to the cost function, but it tends to zero out some features' coefficients, which makes it useful for feature selection.

9. ElasticNet Regression

Linear Regression suffers from overfitting and can't deal with collinear data. When there are many features in the dataset and even some of them are not relevant to the predictive model. This makes the model more complex with a too-inaccurate prediction on the test set (or overfitting). Such a model with high variance does not generalize on the new data. So, to deal with these issues, we include both L-2 and L-1 norm regularization to get the benefits of both Ridge and Lasso at the same time. The resultant model has better predictive power than Lasso. It performs feature selection and also makes the hypothesis simpler.

10. Bayesian Linear Regression

As the name suggests this algorithm is purely based on [Bayes Theorem](#). Because of this reason only we do not use the Least Square method to determine the coefficients of the regression model. So, the technique which is used here to find the model weights and parameters relies on features posterior distribution and this provides an extra stability factor to the regression model which is based on this technique.

2. Regression Metrics

[Regression](#) is a supervised learning technique used to model and analyze the relationship between input variables (features) and a continuous output variable (target). The primary objective of a regression model is to make accurate numerical predictions.

- They quantify the prediction error of regression models
- Different metrics emphasize different error characteristics
- Some metrics penalize large errors more than small ones
- Metric selection depends on problem requirements and data nature
- They help in model selection and optimization

Types of Regression Metrics

1. Mean Absolute Error (MAE)

[Mean Absolute Error](#) measures the average absolute difference between actual and predicted values. It treats all errors equally, regardless of their direction and provides results in the same unit as the target variable, making it easy to interpret.

2. Mean Squared Error (MSE)

[Mean Squared Error](#) calculates the average of squared differences between actual and predicted values. By squaring errors, it penalizes larger mistakes more strongly, making it sensitive to outliers.

3. Root Mean Squared Error (RMSE)

Root Mean Squared Error is the square root of MSE. It maintains the strong penalty for large errors while converting the result back to the original unit of the target variable, improving interpretability.

4. R-squared (R^2 Score)

R-squared represents the proportion of variance in the target variable that is explained by the regression model. It provides insight into how well the model captures underlying data patterns.

5. Mean Absolute Percentage Error (MAPE)

Mean Absolute Percentage Error expresses prediction error as a percentage of the actual values. It is intuitive and scale-independent but becomes unreliable when actual values are close to zero.

Overfitting and Underfitting in Linear Regression

Overfitting

When a model learns each and every pattern and noise in the data to such extent that it affects the performance of the model on the unseen future dataset, it is referred to as **overfitting**. The model fits the data so well that it interprets noise as patterns in the data.

When a model has low bias and higher variance it ends up memorizing the data and causing overfitting. Overfitting causes the model to become specific rather than generic. This usually leads to high training accuracy and very low test accuracy.

Detecting overfitting is useful, but it doesn't solve the actual problem. There are several ways to prevent overfitting, which are stated below:

- Cross-validation
- If the training data is too small to train add more relevant and clean data.
- If the training data is too large, do some feature selection and remove unnecessary features.
- Regularization

Underfitting:

Underfitting is not often discussed as often as overfitting is discussed. When the model fails to learn from the training dataset and is also not able to generalize the test dataset, is referred to as **underfitting**. This type of problem can be very easily detected by the performance metrics.

When a model has high bias and low variance it ends up not generalizing the data and causing underfitting. It is unable to find the hidden underlying patterns from the data. This usually leads to low training accuracy and very low test accuracy. The ways to prevent underfitting are stated below,

- Increase the model complexity
- Increase the number of features in the training data
- Remove noise from the data.

Causes of Overfitting in Polynomial Regression

- **High Polynomial Degrees:** Increasing the degree of a polynomial adds "turning points" (or bends) to the curve. A degree-2 polynomial (quadratic) has one bend, while a degree-20 polynomial allows for 19 bends, giving the model the flexibility to contort and perfectly memorize every training point.
- **Noisy Data:** All real-world data contains some random noise or measurement errors. When the polynomial degree is too high, the curve is forced to reach every single data point, treating noise as a fundamental pattern.
- **Insufficient Data:** If the number of data points is small, a high-degree polynomial will have more parameters to estimate than available observations. The model over-adjusts to local variations, creating wildly erratic, "squiggling" waves between points.

To prevent overfitting, you need to constrain the model's flexibility or give it more information. Here are three highly effective methods to achieve this:

1. Regularization (L1 and L2)

This method adds a mathematical penalty to the loss function based on the size of the model's coefficients.

- **Ridge Regression (L2):** Shrinks coefficients close to zero. This dampens wild oscillations in high-degree polynomial curves.
- **Lasso Regression (L1):** Drives unimportant coefficients exactly to zero. This actively removes unnecessary polynomial terms from your equation.

2. K-Fold Cross-Validation

Instead of relying on a single training set, you split your data into (K) equal parts (folds).

- The model trains on $(K-1)$ folds and tests on the remaining fold.
- ☐ This process repeats (K) times so every data point is used for testing once.
- ☐ It provides an accurate measure of how well your model generalizes to unseen data, helping you select the perfect polynomial degree before deployment

Reference

<https://www.geeksforgeeks.org/machine-learning/linear-regression-real-life-examples/>

<https://www.geeksforgeeks.org/machine-learning/types-of-regression-techniques/>

<https://www.geeksforgeeks.org/machine-learning/regression-metrics/>

<https://medium.com/@samahfetouh7/overfitting-and-underfitting-in-linear-regression-12850458d5f9>